

From RTL to GDSII

TRAINEE ver. Day 2

Authors

Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.
Bobby Rian Dewangga, S.T., M.Eng.
Byan Cahaya Rahman Arvendy

POLYTRON

In collaboration with ICDEC

MODUL IV

Modul *Hand-Ons* Digital IC Design
Polytron Indonesia IC Design Engineer Internship
Program MSIB *Batch* 7

Verilog Exercise dan Verifikasi pada Basys 3 Artix-7 FPGA Board

---

A. Tujuan Pelatihan

Tujuan dari modul ini adalah memberikan pengalaman langsung dalam mengimplementasikan bahasa pemrograman Verilog, serta memverifikasinya menggunakan FPGA Basys 3 Artix-7 FPGA Board. Setelah menyelesaikan modul pelatihan ini, peserta diharapkan mampu:

- Mengimplementasikan konsep *behaviour* ke dalam kode Verilog
- Mensimulasikan desain menggunakan Icarus Verilog (`iverilog`) sebagai langkah awal simulasi dan verifikasi
- Memverifikasi desain pada perangkat keras menggunakan Basys 3 Artix-7 FPGA Board melalui *software* Vivado

B. Overview

Pada modul ini, peserta akan belajar implementasi kode Verilog untuk desain logika digital yang telah dibuat di modul sebelumnya menggunakan *Sum of Product* (SoP). Peserta akan diminta untuk mengimplementasikan kode Verilog yang sebelumnya menggunakan *block design* diubah menggunakan konsep *behaviour*. Setelah perancangan selesai, peserta akan melakukan simulasi awal menggunakan `iverilog` untuk memastikan fungsionalitas bekerja dengan baik. Setelah verifikasi awal berhasil, desain kemudian diunggah ke Basys 3 Artix-7 FPGA Board melalui Vivado untuk implementasi pada perangkat keras.

1. Icarus Verilog

Icarus Verilog (`iverilog`) adalah *tool* simulasi dan kompilasi *open-source* untuk bahasa Verilog HDL. *Tool* ini digunakan untuk mensimulasikan desain digital dan memeriksa fungsionalitasnya sebelum diterapkan ke perangkat keras, seperti FPGA. Icarus Verilog mendukung berbagai fitur Verilog dan sering digunakan dalam pengembangan awal untuk memvalidasi desain sistem digital.

- **Instalasi**
  - ➔ **Instalasi pada Windows**
    - *Download file* .exe pada link berikut https://bleyer.org/icarus/iverilog-v12-20220611-x64_setup.exe
    - Ikuti langkah-langkah instalasi yang ditampilkan pada *Dialog Box Window*

➔ **Instalasi pada Linux (Ubuntu/Debian)**

- Gunakan perintah sebagai berikut:

```
sudo apt update
sudo apt install iverilog
```

➔ **Instalasi pada MacOS**

- Gunakan perintah sebagai berikut:

```
sudo port install iverilog
```

- **Contoh Penggunaan**
  - Mengkompilasi file Verilog `[filename].v` dan testbench `[filename_tb].v` menjadi file output `[output_file]` yang dapat dieksekusi

```
iverilog -o [output_file] [filename].v [filename_tb].v
```

  - Menjalankan simulasi menggunakan file *output* yang dihasilkan dari perintah `iverilog`, menampilkan hasil simulasi

```
vvp [output_file]
```

  - Membuat file .vcd yang merekam sinyal digital selama simulasi dari *file output* hasil kompilasi `iverilog` pada kode *design* verilog sebelumnya

```
vvp [output_file] -o [vcd_file]
```

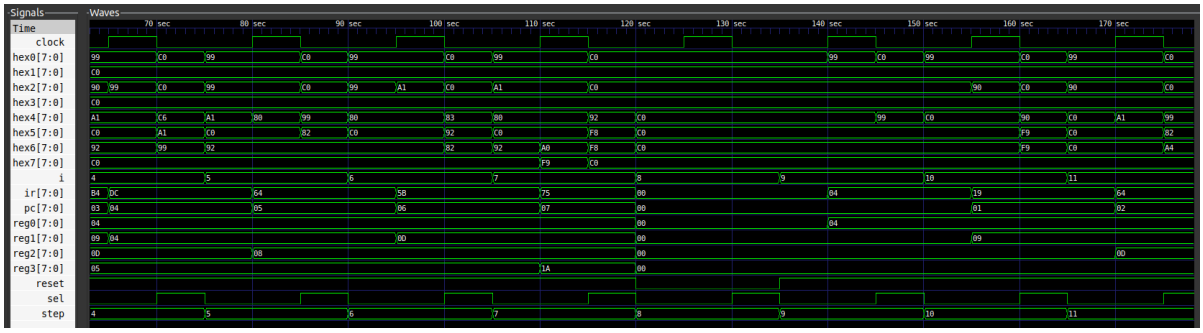
2. GTKWave

GTKWave adalah perangkat lunak *open-source* yang digunakan untuk menampilkan dan menganalisis bentuk gelombang sinyal digital dalam desain sirkuit. *Tool* ini mendukung berbagai format file hasil simulasi, seperti VCD (*Value Change Dump*), LXT, dan FST, yang dihasilkan oleh berbagai simulator digital. GTKWave membantu desainer IC dan perancang sistem digital untuk memvisualisasikan gelombang sinyal digital dari waktu ke waktu, memverifikasi fungsi desain, serta mendeteksi kesalahan logika.

Tool ini mendukung berbagai fitur seperti zoom, pencarian sinyal, dan bookmark, GTKWave memungkinkan pengguna untuk melihat interaksi antara sinyal secara detail, sehingga cocok untuk verifikasi dan simulasi desain RTL dalam proyek desain digital.

Contoh penggunakan *tool* GTKWave adalah sebagai berikut:

```
gtkwave -f [vcd_file]
```



Gambar 1. *Capture* Penggunaan GTKWave untuk melihat *Output* Sinyal Digital dari *Design* yang dibuat

C. *Hands-on* dan *Exercise*

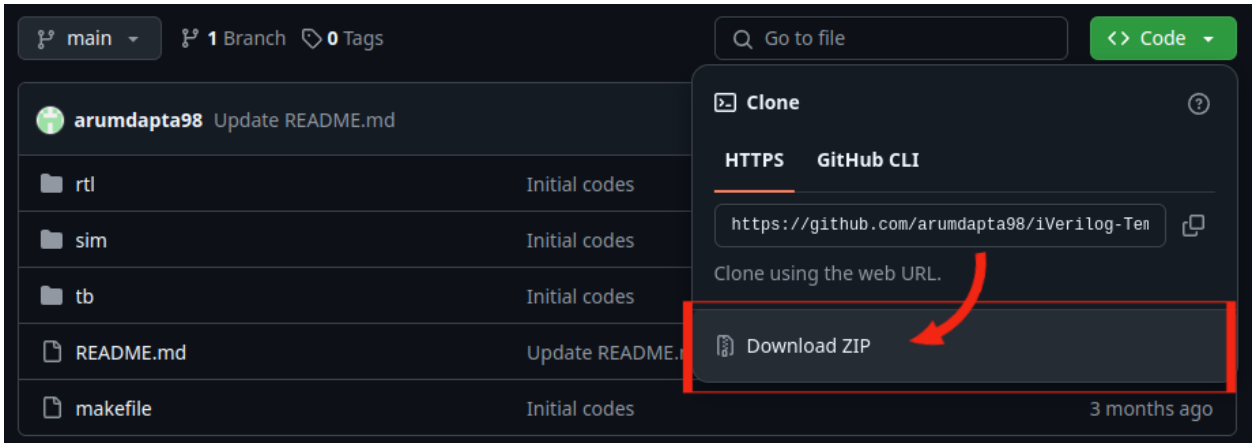
1. Iverilog Template

Dalam projek ini akan digunakan `iverilog` dengan *template project* yang sudah disediakan, untuk memudahkan dalam desain RTL. *Template* `iverilog` dapat di *download* disini : https://github.com/arumdapta98/iVerilog-Template.git

Apabila memiliki `git` dalam komputer lokal, dapat melakukan perintah berikut:

```
git clone https://github.com/arumdapta98/iVerilog-Template.git
```

Namun, apabila tidak memiliki `git` *template* dapat di *download* dengan mengklik pilihan *`Code -> Download ZIP`*.



Dalam *template* ini, terdiri atas hierarki projek sebagai berikut:

```
iVerilog-Template/
 ├── rtl/
 │   └── addsub.v
 ├── sim/
 │   ├── addsub_tb.vcd
 │   └── addsub_tb.vvp
 ├── tb/
 │   └── addsub_tb.v
 ├── README.md
 └── makefile
```

Template ini dibuat untuk memudahkan dalam pengembangan RTL *design*. Komponen utama dari *template* ini terdiri atas 3 direktori yakni `rtl/`, `sim/`, dan `tb/`, serta *file* `README.md/` dan `makefile`. Direktori `rtl/` digunakan sebagai tempat menulis kode Verilog HDL. Direktori `sim/` merupakan direktori simulasi, semua *file* hasil simulasi seperti `.vvp` dan `.vcd` akan disimpan di direktori ini. Sedangkan direktori `tb/` merupakan direktori untuk menyimpan file *test bench* dari desain Verilog yang sudah dirancang. *File* `README.md` *dapat* digunakan untuk menyimpan dokumentasi dari desain RTL yang dirancang. Sedangkan *file* `makefile` digunakan untuk memudahkan dalam proses simulasi RTL.

Dalam *template* ini, peserta dapat menghapus terlebih dahulu *file-file* yang berada di dalam direktori `rtl/`, `sim/`, dan `tb/`. Alternatifnya, peserta juga dapat mengabaikan *file-file* tersebut dan membuat *file* baru sesuai kebutuhan.

2. Mengoperasikan Template Iverilog

Di dalam *project template* ini terdapat *file* `makefile` yang berisi:

```
dir_rtl = rtl
dir_sim = sim
dir_tb  = tb

# define top level module will be tested here!
top_level = addsub

top_level_tb = $(top_level)_tb
simulation_tb = $(dir_sim)/$(top_level_tb)
simulation_tb_vvp = $(simulation_tb).vvp
simulation_tb_vcd = $(simulation_tb).vcd

vvp:
    iverilog -o $(simulation_tb_vvp)  $(dir_tb)/$(top_level_tb).v
```

```
vcd:
    vvp $(simulation_tb_vvp) -o $(simulation_tb_vcd)

gtk:
    gtkwave -f $(simulation_tb_vcd)

all:
    iverilog -o $(simulation_tb_vvp)  tb/$(top_level_tb).v
    vvp $(simulation_tb_vvp) -o $(simulation_tb_vcd)

clean:
    cd $(dir_sim) &     del *.vvp *.vcd
```

File tersebut merupakan *file* `MakeFile`. `Makefile` adalah *file* konfigurasi yang digunakan oleh program *`make`* untuk mengotomatisasi *build* proses pada proyek pemrograman. Dalam isi `Makefile` diatas terdapat beberapa perintah *`make`* yang dapat dilakukan:

- Menggunakan *`iverilog`* untuk mengkompilasi *file test bench* menjadi file *`.vvp`* yang bisa dijalankan.

  ```
  make vvp
  ```

- Menjalankan *file `.vvp`* untuk menghasilkan *wave file `.vcd`*.

  ```
  make vcd
  ```

- Membuka *file `.vcd`* di aplikasi `gtkwave` untuk *wave analysis*.

  ```
  make gtk
  ```

- Kombinasi perintah `vvp` dan `vcd` untuk mengkompilasi dan menjalankan simulasi sekaligus.

  ```
  make all
  ```

- Membersihkan file `.vvp` dan `.vcd` dari direktori simulasi (`sim/`) menggunakan perintah `del`.

  ```
  make clean
  ```

Untuk menggunakan *template* MakeFile ini, peserta dapat terlebih dahulu mengubah bagian *`top_level = addsub`* menjadi *`top_level = [top_module]`*. Ubah bagian `addsub` menjadi nama *design topmodule* verilog yang sesuai dengan *design* yang telah dibuat.

** note :*

- *Dalam menggunakan template project ini, diperlukan binary dari program make. Sehingga dihimbau bagi para peserta yang belum memiliki program make di komputer lokal, harap dapat mendownload program make terlebih dahulu.*

3. Tugas

Buat Verilog untuk mensimulasikan 7-*segment display* kemudian lakukan kompilasi dengan menggunakan `iverilog` dan amati perubahan sinyalnya!

4. Flash pada FPGA Menggunakan Vivado Xilinx

Gunakan *Tools* Vivado Xilinx untuk memverifikasi kode verilog yang sebelumnya Anda buat pada Tugas 1 kemudian uji cobalah kode tersebut pada *board* FPGA Basys 3. Ganti *wiring* IC 7447 menggunakan pin-pin pada Basys 3, kemudian ujilah apakah 7-*segment display* menyala dengan sesuai apa tidak!

5. Tugas Tambahan:

Buatlah folder baru untuk menyimpan file - file verilog pada Tugas Tambahan (poin 5) ini. Kemudian, lakukan perancangan dengan melakukan modifikasi pada kode Verilog pada poin 3 di atas sehingga nantinya, angka desimal 0-9 dapat berpindah antar display 7-segment yang ada pada FPGA board Basys 3 (total ada 4 display) berdasarkan masukan switch. Misal, apabila switch 3 bernilai HIGH, maka display 7-segmen yang hidup adalah yang ketiga dari kanan.

- Lakukan simulasi di iverilog untuk melihat apakah masukan switch di atas sudah bekerja (dan juga menampilkan angka dari 0-9 seperti pada poin 3).
- Kemudian, lakukan flash pada FPGA board Basys 3 untuk melihat bahwa rancangan yang sudah Anda buat benar-benar berjalan sesuai instruksi di atas.